

Algorithmique Avancée

Examen Réparti 2

AVEC CORRECTION

Les seuls documents autorisés sont les polys de cours, ainsi que la copie double personnelle. Le barème donné est indicatif.

Exercice 1 : QCM [6 points]

Dans ce QCM, pour chaque question vous devez donner 1 seule réponse et expliquer votre choix par une ligne de texte ou une figure. Le barème est le suivant : **Réponse correcte et correctement argumentée : 1 point. Réponse incorrecte ou correcte mais mal argumentée : 0 point. Il n'y a pas de points négatifs.**

Question 1 On considère une table de hachage de taille $2N$ dans laquelle on insère $N/8$ clés, avec une fonction de hachage uniforme. Le nombre moyen de clés qui ont la même valeur de hachage i fixée est
A. $1/16$. **B.** $1/4$. **C.** $1/2$.

Question 2 Pour un arbre 2-3-4 contenant n clés, l'algorithme permettant de calculer la hauteur de l'arbre est au pire cas (en nombre de nœuds traversés)
A. $\Theta(n)$. **B.** $\Theta(\log n)$. **C.** $\Theta(1)$.

Question 3 Pour un problème NP-complet.
A. Il y a des instances (entrées) qui n'ont pas de solution.
B. Pour chaque instance, on ne sait pas calculer une solution en temps raisonnable.
C. Il existe une famille d'instances difficiles à résoudre.

Question 4 Un filtre de Bloom

- A.** est une méthode de hachage.
- B.** permet de savoir si une clé est insérée dans une table.
- C.** permet de savoir si une clé n'est pas insérée dans une table.

Question 5 Laquelle de ces 3 suites d'inégalités d'ordres de grandeur est correcte, c'est-à-dire correcte à partir d'un certain rang $n \in \mathbb{N}$ (expliquer pourquoi les autres sont incorrectes)

- A.** $3^n < n^2 2^n < n! < n^{10n}$.
- B.** $n^4 \log n < n^5 < n^4 2^n$.
- C.** $2^n < n! < (\sqrt{n})^n$.

Question 6 L'arbre binomial B_k

- A.** a une profondeur $\Theta(\log(k))$.
- B.** a sa racine de degré (i.e. d'arité) k .
- C.** a ses fils gauche et droit qui sont des arbres binomiaux B_{k-1} .

Solution

1. **A.** car $(N/8)/(2N)$
2. **B.** car toutes les feuilles sont au même niveau
3. **C.** car les autres possibilités sont fausses;-)
4. **C.** il y a des faux-positifs (mais pas de faux négatifs)
5. **B.** pas **A.** car $3^n > n^2 2^n$ et pas **C.** car Stirling
6. **B.** cours

Exercice 2 : Hachage et tri [6 points]

On a un tableau T contenant des numéros d'étudiants à 7 chiffres que l'on souhaite trier dans l'ordre croissant.

On utilise pour cela des tables de hachage avec chaînage, avec insertion dans les listes (gérant les collisions) *en queue de liste*.

On a accès à une fonction `chiffre` prenant en entrée un numéro d'étudiant et un chiffre i et renvoyant le i -ème chiffre du numéro en partant de la droite. Par exemple `chiffre(1234567, 1) = 7` et `chiffre(1234567, 3) = 5`.

XXX il faut améliorer un peu la rédaction : lien entre `chiffre` et la fonction de hachage

Voilà l'algorithme de tri :

Soit d un tableau contenant tous les numéros d'étudiants.

Pour i allant de 1 à 7 faire

 Soit t une table de hachage de taille 10 dont

 la fonction de hachage est $h(k) = \text{le } i\text{-ème chiffre de } k$

 Pour chaque numéro k de d Faire

 Ajouter k à t

 Fin Pour

Vider d

 Ranger tous les numéros de t dans le tableau d en parcourant
 les cases de 0 à 9 et les listes dans l'ordre

Fin Pour

Afficher le contenu de d

Question 1 On va tester l'algorithme sur la liste d composées des numéros suivants : 3200231, 3200045, 3600529, 3600009, 3300067, 3333333, 3600051, 3600021. Pour chaque étape i , dessiner la table t et le tableau d .

Question 2 Prouver la correction de l'algorithme.

Question 3 Que se passerait-il si l'insertion dans les listes se faisait en tête plutôt qu'en queue de liste ?

Question 4 Après avoir indiqué la mesure de complexité adéquate, calculer la complexité en temps de l'algorithme.

Question 5 Comment modifier l'algorithme pour trier les numéros dans l'ordre décroissant ?

Solution

1. Voilà le contenu de d

[3200231, 3600051, 3600021, 3333333, 3200045, 3300067, 3600529, 3600009]
[3600009, 3600021, 3600529, 3200231, 3333333, 3200045, 3600051, 3300067]
[3600009, 3600021, 3200045, 3600051, 3300067, 3200231, 3333333, 3600529]
[3600009, 3600021, 3200045, 3600051, 3300067, 3200231, 3600529, 3333333]
[3600009, 3600021, 3200045, 3600051, 3300067, 3200231, 3600529, 3333333]
[3200045, 3200231, 3300067, 3333333, 3600009, 3600021, 3600051, 3600529]
[3200045, 3200231, 3300067, 3333333, 3600009, 3600021, 3600051, 3600529]

2. Invariant de boucle : A chaque fin de tour de la boucle sur i : Les numéros sont classés dans l'ordre croissant on ne considérant que les i plus petits chiffres de chaque numéro.

3. insertion en tête dans t :

[3600021, 3600051, 3200231, 3333333, 3200045, 3300067, 3600009, 3600529]
[3600009, 3600529, 3600021, 3333333, 3200231, 3200045, 3600051, 3300067]
[3300067, 3600051, 3200045, 3600021, 3600009, 3200231, 3333333, 3600529]
[3600529, 3200231, 3600009, 3600021, 3200045, 3600051, 3300067, 3333333]
[3300067, 3600051, 3200045, 3600021, 3600009, 3200231, 3600529, 3333333]
[3200231, 3200045, 3333333, 3300067, 3600529, 3600009, 3600021, 3600051]
[3600051, 3600021, 3600009, 3600529, 3300067, 3333333, 3200045, 3200231]

A l'étape i , les numéros dont le i -ème

chiffre sont le même sont triés dans l'ordre décroissant. Mais au fur et à mesure qu'on avance dans l'algo, il y a un "mélange" qui s'opère.

4. Complexité en nombre d'écriture de numéros en mémoire (par exemple) :

On suppose que d contient n numéros.

A chaque tour on écrit n numéros dans t et autant dans d .

Donc la complexité de l'algorithme vaut $14n = O(n)$.

5. A chaque étape, on remplit d en parcourant les cases de 9 à 0.

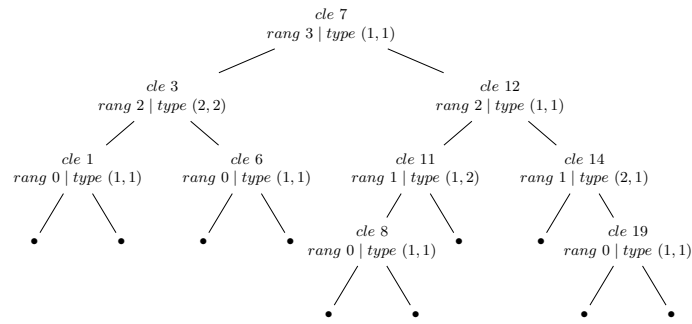
Exercice 1 : Arbre de rang équilibré (WAVL) [13 points]

On introduit la famille d'arbres binaires de recherche appelés les arbres de rang équilibré (WAVL). Ce sont des structures proches des arbres de hauteur équilibrée (AVL). En fait, ils sont un peu moins contraints.

Par la suite, un arbre binaire est composé de nœuds internes binaires (avec exactement deux fils) et de nœuds externes vides. Chaque nœud interne binaire contient une clé. On appelle feuille de l'arbre, un nœud interne dont les deux fils sont vides : par exemple, dans l'arbre ci-contre, le nœud contenant la clé 6 est une feuille.

À chaque nœud x , on associe un entier appelé son *rang* et noté $r(x)$. Le rang d'un nœud (autre que la racine) est strictement inférieur au rang de son père. Par définition, si x est une feuille alors $r(x) = 0$ et si c'est un nœud vide alors $r(x) = -1$. Pour un nœud x , père de deux feuilles, on peut définir son rang comme étant 1, 2 ou tout autre entier strictement positif.

Pour chaque nœud x (autre que la racine), on définit sa *différence de rang* comme étant l'entier $r(\text{pere}(x)) - r(x)$, et on la note $\Delta(x)$. Pour un nœud interne x , on note x_g (resp. x_d) son sous-arbre gauche (resp. droit). À chaque nœud interne x , on associe son *type* $(\Delta(x_g), \Delta(x_d))$. Par exemple, une feuille est de type $(1, 1)$.



Un WAVL est un arbre binaire de recherche vérifiant les contraintes suivantes :

- La différence de rang d'un nœud (autre que la racine) vaut 1 ou 2.
- Les feuilles sont de rang 0.

Question 1 Construire 2 nouveaux WAVL dont la structure arborescente et la disposition des clés sont les mêmes que dans l'exemple ci-dessus (il faut simplement modifier des rangs et/ou des types de certains nœuds).

Question 2 Montrer que pour un WAVL ne contenant pas de nœud de type $(2, 2)$, le rang d'un nœud correspond à sa hauteur. On rappelle que la hauteur d'un arbre est la plus grande longueur de chemin entre la racine et les feuilles.

Question 3 À quelle famille les WAVL ne contenant pas de nœud de type $(2, 2)$ correspondent-ils ?

Dans la suite, utiliser les primitives qui semblent nécessaires pour décrire les algorithmes (**père(x)**, **fils-gauche(x)**, **rang(x)**, **delta(x)**, ...).

Remarque : Ne pas oublier que les nœuds ont un rang à tenir à jour.

Question 4 Donner le pseudo-code de l'algorithme **EstDans** qui prend en arguments une clé et un WAVL et renvoie **vrai** si la clé est dans le WAVL et **faux** sinon.

L'insertion d'un nouvel élément se fait "au bon endroit" (un WAVL est un arbre binaire de recherche), en transformant une feuille en nœud binaire et en lui rattachant deux nouvelles feuilles.

Question 5 Donner le pseudo-code de l'algorithme **Insertion** qui réalise l'insertion d'une clé dans un WAVL. On supposera que cette clé n'était pas déjà présente.

Attention : L'arbre résultant de l'insertion n'est plus forcément un WAVL, mais il n'est pas demandé pas de le rééquilibrer dans cette question.

Question 6 Rappeler, à l'aide d'un dessin ou d'un pseudo-code, la définition de la rotation gauche et de la double rotation gauche-droite dans arbre binaire.

On note qu'il existe aussi les rotations droite et droite-gauche, mais il n'est pas demandé de les rappeler.

Comme dans le cas d'un arbre de hauteur équilibrée (AVL), après l'insertion d'un nœud, on doit rééquilibrer le WAVL pour se conformer à la contrainte sur les rangs de ses nœuds. Pour rééquilibrer un WAVL après l'insertion d'un nœud x , on procède ainsi : *tant que le père de x existe, on met à jour son type (de ce père) et si il est de type $(1, 0)$ ou $(0, 1)$ alors on incrémente son rang, puis on itère avec $x \leftarrow \text{pere}(x)$.*

À la fin de cette boucle, si les contraintes de rang sont respectées, le WAVL est équilibré. Sinon, le père de x est de type $(0, 2)$ ou $(2, 0)$. Dans ce cas, si le frère de x est un nœud vide ou un nœud avec deux fils non vides, une rotation simple le rééquilibre. Sinon, une rotation double est nécessaire. Mettre à jour les rangs des nœuds (par incrémentation et décrémentation).

Question 7 En utilisant le WAVL exemple (en haut à droite de cette page), insérer successivement les clés 20, 21, 22 et 13. Dessiner successivement chacun des WAVL obtenus (après l'insertion et le rééquilibrage de chacune des clés).

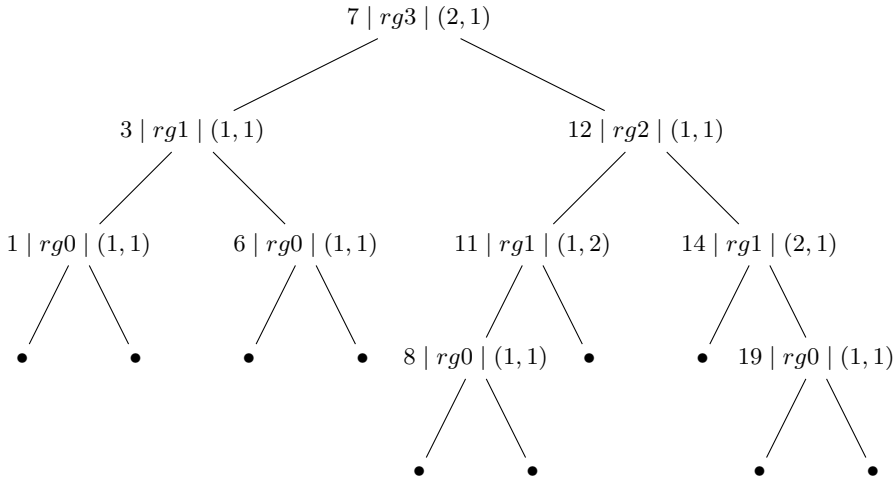
Question 8 Par induction, montrez que si k , h sont respectivement le rang et la hauteur d'un WAVL alors $h \leq k \leq 2h$.

Question 9 Montrez que si k et n sont respectivement le rang et le nombre de nœuds non vides d'un WAVL alors $k \leq 2 \log_2 n$. **Indication :** Posez la récurrence de la suite (n_k) de la taille minimale d'un WAVL de rang k .

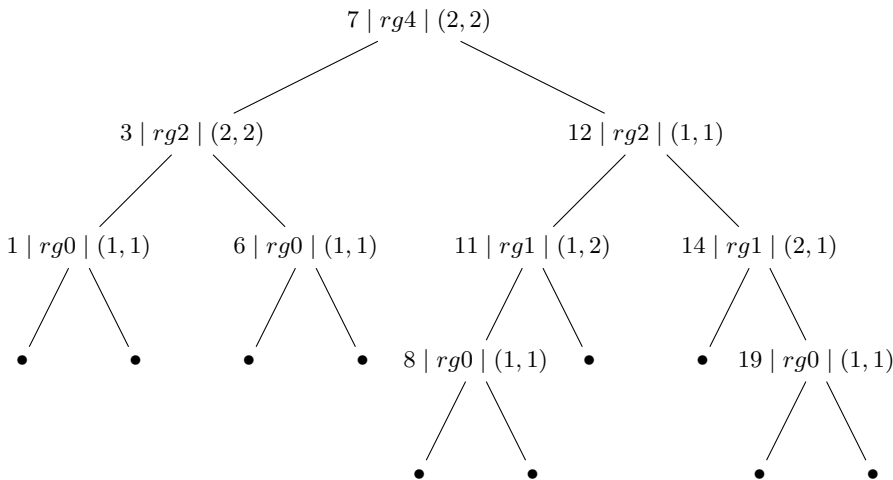
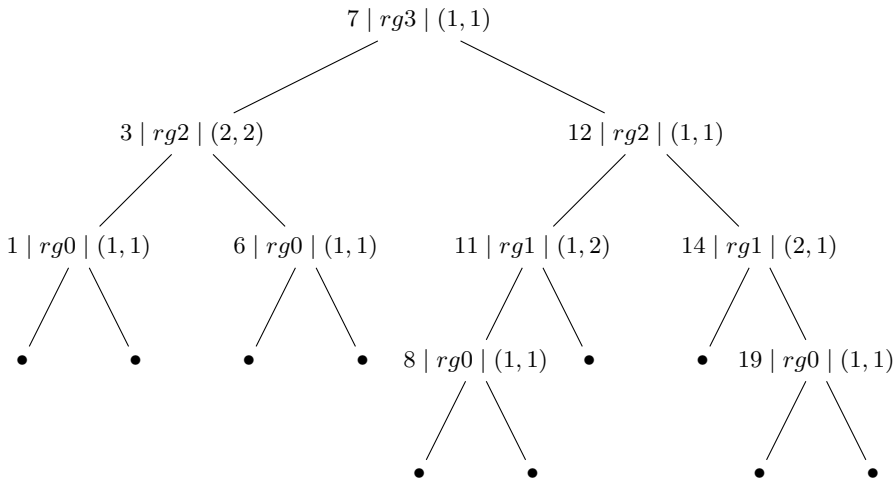
Question 10 En utilisant les questions précédentes, donnez la complexité au pire cas, en nombre d'incrémentations, d'un équilibrage dans un WAVL ?

Question 11 On définit le potentiel d'un nœud comme étant 1 si c'est un nœud (non feuille) de type $(1, 1)$, $(0, 1)$ ou $(1, 0)$ et 0 sinon. On définit la fonction de potentiel Ψ d'un arbre comme la somme des potentiels de ses nœuds. En utilisant la méthode du potentiel avec Ψ , calculer la complexité amortie, en nombre d'incrémentations, de l'insertion.

Solution



1.



2. On prouve par induction. Cas de base : le WAVL réduit à une feuille a rang 0 ce qui correspond à sa hauteur. Hypothèse d'induction : il existe n tel que pour tout WAVL de taille n la hauteur de ses WAVL est égal à leur rang. Soit T un WAVL de taille $n + 1$ sans nœud de type $(2, 2)$, de racine x , de fils gauche G (de rang g) et fils droit D (de rang d). On distingue plusieurs cas :

- si $g = d$: alors le rang de x est nécessairement $g + 1$ et non $g + 2$ sinon G et D auraient des racines de type $(2, 2)$. Par hypothèse d'induction g (resp. d) hauteur de G (resp. D), donc $g + 1$ est bien la hauteur de T . Qed.
- si $g < d$: nécessairement $g + 1 = d$ et le rang x est donc $d + 1$. Par hypothèse d'induction, d est la hauteur de D et donc $d + 1$ est la hauteur de T . Qed.
- si $g > d$: symétrique du cas précédent.

3. Les différences de hauteur dans les WAVL sans nœud de type $(2, 2)$ est d'au plus 1 donc ce sont des AVL.

4. `def EstDans(T,x):`
 si $T = \text{arbre-vide}$:
 retourner Faux
 si $\text{racine}(T) = x$:
 retourner Vrai

```

si racine(T) < x:
    retourner EstDans(fils-gauche(T), x)
sinon:
    retourner EstDans(fils-droit(T), x)

```

```

5. def Insertion(T,x):
    si T = arbre-vide:
        f = Feuille(x)
        rang(f) = 0
        retourner f
    si racine(T) <= x:
        retourner Insertion(fils-gauche(T), x)
    sinon:
        retourner Insertion(fils-droit(T), x)

```

```

6. def rotation-gauche(T):
    x = racine(T)
    y = racine(fils-droit(T))
    A = fils-gauche(fils-droit(T))
    B = fils-droit(fils-droit(T))

    retourner Arbre(y, Arbre(x, fils-gauche(T), A), B)

```

```

def rotation-droite(T):
    x = racine(T)
    y = racine(fils-gauche(T))
    A = fils-gauche(fils-gauche(T))
    B = fils-droit(fils-droit(T))

    retourner Arbre(y, A, Arbre(x, B, fils-droit(T)))

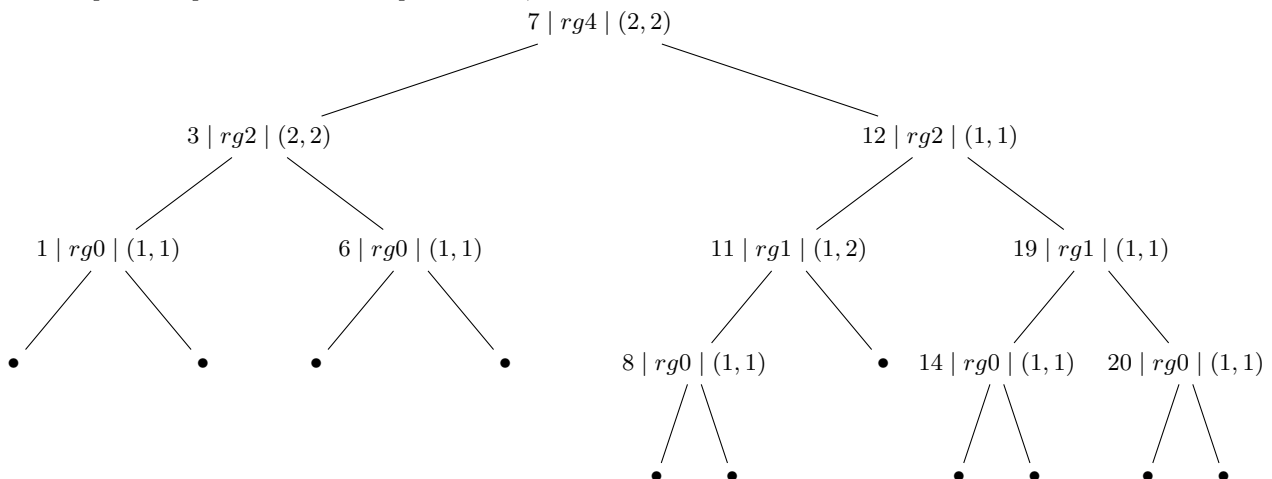
```

```

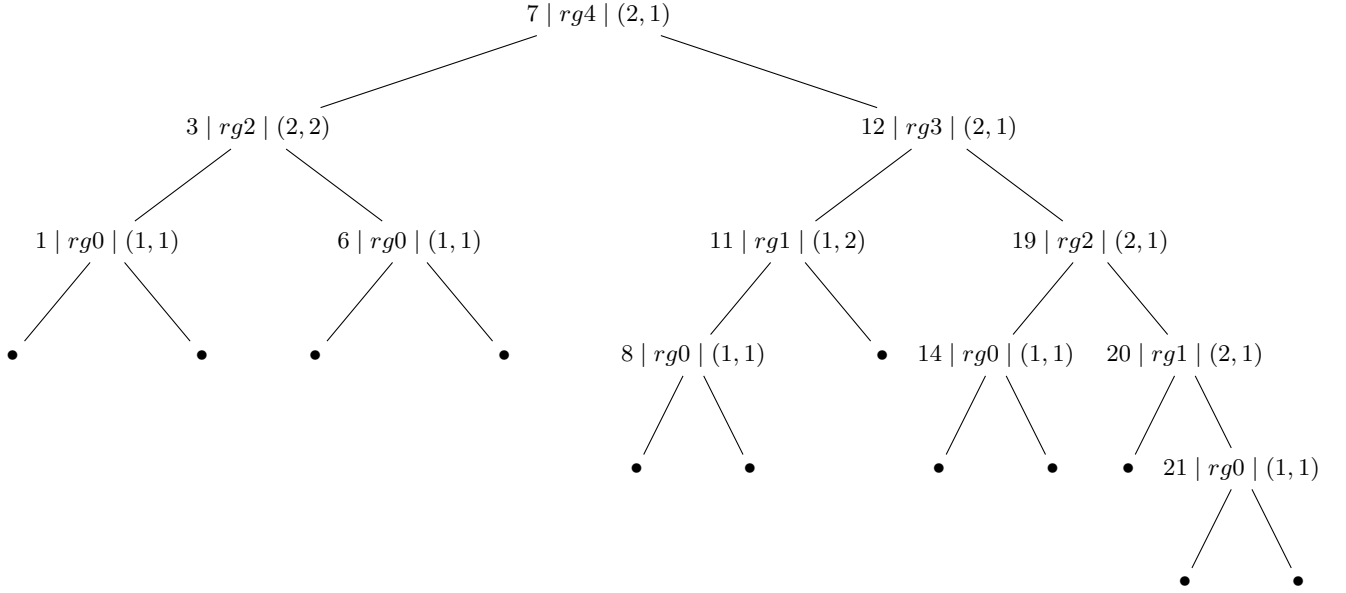
def rotation-gauche-droite(T):
    retourner rotation-droite(
        Arbre(racine(T),
            rotation-gauche(fils-gauche(T)),
            fils-droit(T)))

```

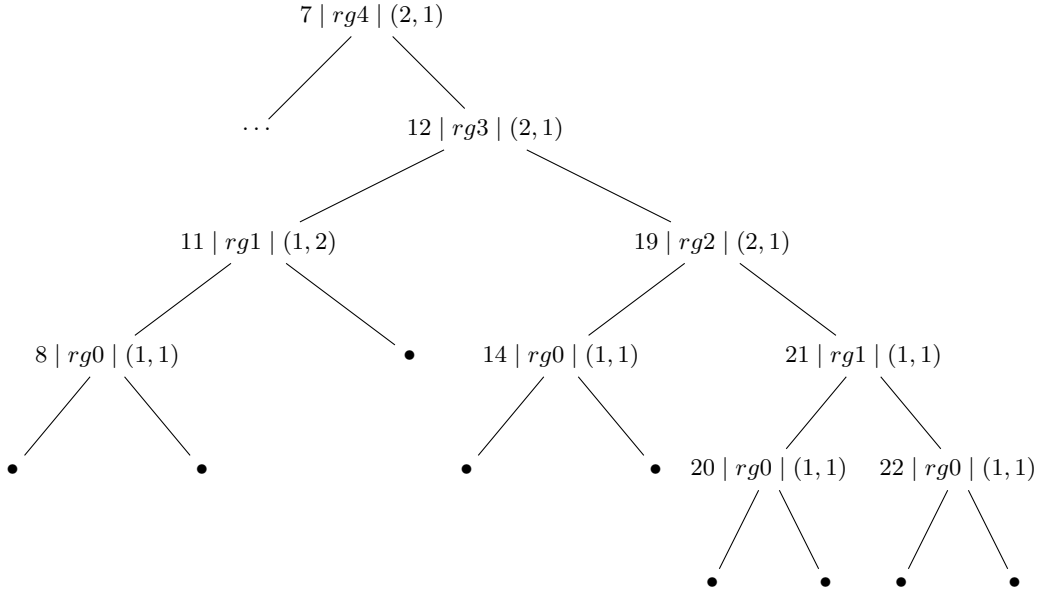
7. C'est pareil pour les 3 possibilités, seul le sous arbre enraciné en 14 est modifié :



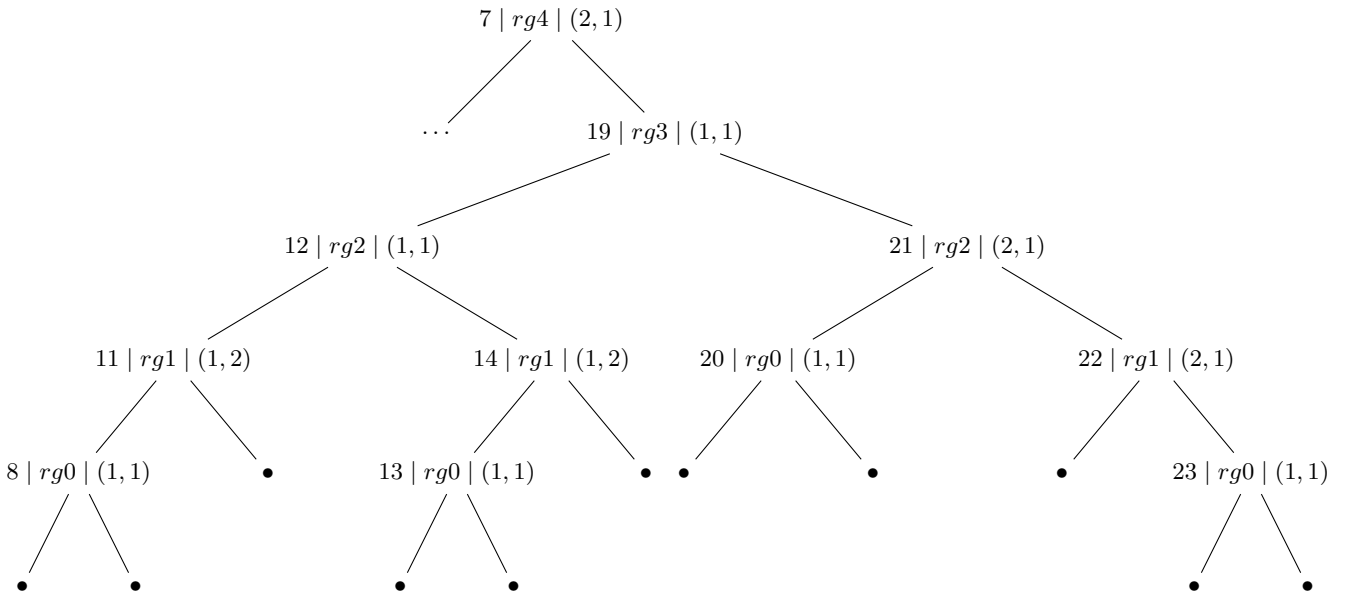
insertion de 21 :



insertion de 22 (le sous arbre gauche n'est pas modifié) :



insertion de 13 (le sous arbre gauche n'est pas modifié) :



8. direct par induction

9. poser $n_k = 1 + 2n_{k-2}$ en déduire $n_k \geq 2^{\lfloor k/2 \rfloor}$ et donc $k \leq 2 \log_2 n$

10. Corollaire direct des deux questions précédentes.

11. On distingue différents cas d'insertions :

- insertion dans un arbre vide $\Rightarrow \hat{c} = 0$
 - insertion comme frère d'une feuille déjà existante : pas de rééquilibrage donc pas d'incrémentations mais création d'un nœud $(1, 1) \Rightarrow \hat{c} = 0 + \Psi(T_{apres}) - \Psi(T_{avant}) = 1$
 - insertion avec rééquilibrage : fait un nombre k d'incrémentations sur des nœuds de types $(0, 1)$ ou $(1, 0)$, avant de s'arrêter sur un nœud (la racine ou un nœud de type $(0, 2)$). Au pire, il faut faire une rotation double qui coûte deux incrémentations. D'où $\hat{c} \leq k$ incrémentations + la dernière + 2 dans les rotations + $\Psi(T_{apres}) - \Psi(T_{avant}) = k + 1 + 2 + (\Psi(T_{avant}) - k) - \Psi(T_{avant}) = 3$
- On conclut que le cout amortie d'une insertion est d'au plus 3.